Count-Min Sketch

Frequency Estimation in Large Data Streams

Daksh Shaurya Swapnil Tridham Akshay Shinjan G

Indian Statistical Institute

Frequency estimation using hashing, compact counters, and probabilistic error guarantees

Presentation Roadmap

1. Problem Scenario
2. Hash Functions
3. Count-Min Sketch
4. Probabilistic Analysis
5. How the Stream Affects the Sketch
6. Worked Example and Numerical Check
7. Applications
8. Limitations and Conclusion

Central Question

How can we estimate the frequency of an item in a massive data stream while using substantially less memory than exact counting?

What Is a Data Stream?

A **data stream** is an ordered sequence of observations or events that arrive over time:
$x_1, x_2, x_3, \ldots$
Unlike a small static dataset, a stream may be too large, too fast, or too continuous to store completely.

Why the whole stream may not be available

- **Too large:** billions or trillions of records.
- **Continuous:** new observations may never stop.
- **High velocity:** data arrives too quickly to store first.
- **One-pass processing:** an item may need to be processed and discarded immediately.

Typical examples

- Network packets
- Search queries
- Financial transactions
- Website clicks
- Sensors
- System logs

Why Data Streams Are Different

For a small static dataset, we can store every observation. For a massive stream, this may be impossible.

$$\boxed{\text{Continuous arrival} \Longrightarrow \text{memory cannot grow indefinitely}}$$

- The stream may contain billions of observations.
- The number of distinct items may also be enormous.
- Storing the complete stream may be unnecessary.
- Often we need only summary information.

Therefore, streaming algorithms summarize the stream using **limited memory**.

Key feature

Data is processed as it arrives, often in a single pass, without keeping the historical stream in memory.

The Frequency Question

Consider the stream $A, B, A, C, A, B, D, A, \ldots$
A natural question is:

Frequency Query

How many times has a particular item appeared in the stream?

For an item x, $f(x) = \#\{i : x_i = x\}$. For example, $f(A) = 4$ in the displayed prefix.
The challenge is to estimate $f(x)$ without storing an enormous frequency table.

Exact Frequency Counting

The obvious solution is an exact frequency table. For every distinct item x, store $x \longrightarrow f(x)$.

Item	A	B	C	D	E
Frequency	5	3	2	2	1

Every time an item arrives, its corresponding counter is increased.

Problem

We need a separate entry for every distinct item.

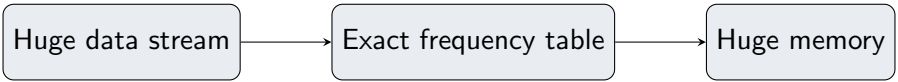
If there are millions or billions of distinct items, this requires a very large amount of memory.

The Memory Problem

The basic relationship is

$$\text{more distinct items} \Longrightarrow \text{more counters} \Longrightarrow \text{greater memory requirement.}$$

Suppose a stream contains 10^9 distinct identifiers. An exact table needs roughly 10^9 separate counters, even if we are interested in only a few queries.



From Exactness to Approximation

Instead of storing every frequency exactly, construct a compact summary of the stream. The problem changes from

How can every frequency be stored exactly?

to

How can frequencies be estimated using limited memory?

This introduces a trade-off: **Less memory $\iff$ Approximation.**
But approximation is useful only if its error can be controlled. This is where probability becomes important.

Why Hash Functions?

Hash function

A function that takes an item as input and maps it to a fixed range of bucket positions, so that the item can be assigned to a specific counter in the sketch.

A compact frequency summary needs to map arbitrary items into a small number of counters:

$$\text{item} \longrightarrow \text{hash function} \longrightarrow \text{bucket}$$

For a row containing w buckets, $h(x) \in \{1, \ldots, w\}$. Different items may map to the same bucket; that event is called a **collision**.

Hash Functions as Black Boxes

For our analysis, we do not need to know how the hash function is implemented internally. We treat it as a black box:

$$x \longrightarrow \boxed{h} \longrightarrow j, \qquad j \in \{1, \ldots, w\}.$$

What matters are its statistical properties:

1. Uniformity
2. Low collision probability
3. Independence across rows

These properties determine the quality of the frequency estimate.

Uniformity

A desirable hash function distributes items approximately uniformly among the w buckets. Ideally,

$$\mathbb{P}(h(x) = j) = \frac{1}{w} \quad \text{for every bucket } j.$$

Thus each bucket is equally likely to receive a given item.

- No bucket is systematically preferred.
- Counters receive a balanced distribution.
- Excessive collisions become less likely.

Uniformity is the ideal intuition behind the analysis. The proof itself needs only the pairwise collision property (next slides), not full uniformity.

Collisions

A collision occurs when two distinct items map to the same bucket: $x \neq y,\ h(x) = h(y)$. For example, $A \to 3$ and $B \to 3$. Then the counter at bucket 3 contains contributions from both A and B.

Collisions are unavoidable when the number of possible items is much larger than the number of buckets.

The objective is therefore not to eliminate collisions, but to **control their effect**.

Pairwise Collision Property

For distinct items $x \neq y$, assume that in row j,

$$\mathbb{P}\big(h_j(x) = h_j(y)\big) = \frac{1}{w}.$$

This is the key within-row property used in the expectation analysis.

Independence Across Rows

Count-Min Sketch uses multiple hash functions: $h_1, h_2, \ldots, h_d$. The hash functions of different rows are assumed independent.
If x collides badly in one row, a different row uses a different hash function and may have a completely different collision pattern.

$$\boxed{\text{Independent rows} \implies \text{bad collisions are unlikely in every row}}$$

This independence allows failure probabilities to multiply.

What Is Count-Min Sketch?

Count-Min Sketch (CMS) is a probabilistic data structure for estimating the frequency of items in a data stream.
Instead of storing item $\rightarrow$ exact frequency, CMS stores a small matrix of counters

$$C \in \mathbb{N}^{d \times w}.$$

Each row has its own hash function $h_1, h_2, \ldots, h_d$.

Count-Min Sketch: Update and Query

When an item x arrives, CMS increments one counter in every row:

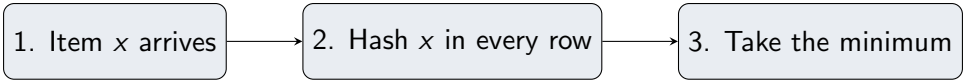
$$C[j, h_j(x)] \leftarrow C[j, h_j(x)] + 1.$$

To query x, read those d counters and return their minimum:

$$\hat{f}(x) = \min_{1 \le j \le d} C[j, h_j(x)]$$

Count-Min Sketch: The Core Idea

The entire algorithm can be viewed as three steps.



More explicitly,

$$x \xrightarrow{h_1, h_2, \ldots, h_d} \big(C[1, h_1(x)], C[2, h_2(x)], \ldots, C[d, h_d(x)]\big), \qquad \hat{f}(x) = \min_j C[j, h_j(x)].$$

This gives a fixed-size summary of an arbitrarily long stream.

Why Is It Called "Count-Min Sketch"?

The name describes exactly what the structure does.

Count

Each incoming item increments counters: $C[j, h_j(x)] \mathrel{+}= 1$.

Min

A query reads one counter per row and takes the minimum: $\hat{f}(x) = \min_j C[j, h_j(x)]$.

Sketch

Only a compact summary is stored, not the full frequency table.

Why Does Count-Min Sketch Work?

For a target item x, every occurrence of x increments its corresponding counter in every row. Therefore each relevant counter contains at least $f(x)$.
Other items may collide with x, adding extra counts. Hence every row gives

$$\text{true frequency} + \text{collision noise}.$$

So the j-th row gives $C[j, h_j(x)] = f(x) + X_j$ with $X_j \geq 0$. Taking the minimum chooses the row with the smallest collision noise:

$$\hat{f}(x) = f(x) + \min_j X_j.$$

This is the central idea behind CMS.

Why Is It Called a Sketch?

A photograph preserves enormous detail. A sketch preserves only the information necessary to capture the main structure. Count-Min Sketch behaves similarly.

- It does not preserve the complete stream.
- It does not store an exact entry for every item.
- It preserves enough aggregate information to estimate frequencies.

Thus *compact representation + controlled approximation* is the central philosophy.

Structure of the Sketch

The sketch is a $d \times w$ table $C[j, k]$, $1 \leq j \leq d$, $1 \leq k \leq w$. Here:

- d: number of rows;
- w: number of buckets per row;
- h_j: hash function associated with row j.

Total number of counters: dw. This number is fixed once the sketch is constructed.

An Empty Count-Min Sketch

For example, with $d = 4$ and $w = 6$:

	1	2	3	4	5	6
Row 1	0	0	0	0	0	0
Row 2	0	0	0	0	0	0
Row 3	0	0	0	0	0	0
Row 4	0	0	0	0	0	0

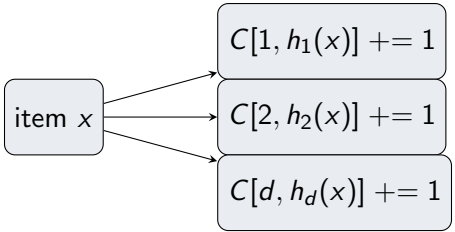
Initially $C[j, k] = 0$ for every j, k. Each incoming item increments exactly one counter in every row.

Operation 1: Updating the Sketch

Suppose item x arrives. For every row j,

$$C[j, h_j(x)] \leftarrow C[j, h_j(x)] + 1.$$

Thus one occurrence of x produces exactly d counter increments.



The item itself does not need to be stored.

Operation 2: Querying a Frequency

To query item x, look at its corresponding counter in every row: $C[1, h_1(x)], \ldots, C[d, h_d(x)]$. Then return

$$\hat{f}(x) = \min_{1 \le j \le d} C[j, h_j(x)].$$

For example, if the counters are $7, 5, 8, 5$, then $\hat{f}(x) = 5$. Hence the name: Count-Min Sketch.

Why the Minimum?

Suppose the true frequency is $f(x) = 5$. Because every occurrence of x increments its corresponding counter, every row contains at least 5. Collisions can only add extra counts:

$$7, \quad 5, \quad 8, \quad 6.$$

The smallest value is therefore the least contaminated measurement:

$$\hat{f}(x) = \min(7, 5, 8, 6) = 5.$$

Key intuition

One clean row is enough to obtain a good upper bound.

One-Sided Error

For non-negative updates, $C[j, h_j(x)] \geq f(x)$ for every row j. Therefore

$$\hat{f}(x) \geq f(x) \quad \text{always.}$$

So Count-Min Sketch does not underestimate the frequency under the standard non-negative-update model. The only source of error is additional mass caused by collisions:

$$\hat{f}(x) - f(x) = \text{collision contribution.}$$

Complexity and Memory

For each arriving item we perform one update per row: **update time** $= O(d)$. For a query we inspect one counter per row: **query time** $= O(d)$.
The sketch stores $d \times w$ counters: **memory** $= O(dw)$. Later we choose

$$w = O(1/\varepsilon), \qquad d = O(\ln(1/\delta)).$$

Thus the number of counters, $O\big(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\big)$, does not depend on the number of distinct items.

From the Sketch to Probability: the Collision Error

So far the sketch is built, updated and queried. Now the hash functions are **random**. Fix a target item x and a row j:

$$X_j = C[j, h_j(x)] - f(x) = \sum_{y \neq x} f(y)\, \mathbf{1}\{h_j(y) = h_j(x)\} \;\geq 0.$$

Here $\mathbf{1}\{h_j(y) = h_j(x)\} = 1$ if y lands in the same bucket as x, and 0 otherwise.

Meaning

X_j = total frequency of the *other* items sharing x's bucket in row j. Hence $\hat{f}(x) = f(x) + \min_j X_j$.

Expected Collision Error

Let $N = \sum_y f(y) = \|f\|_1$. By linearity of expectation and the pairwise property $\mathbb{P}(h_j(y) = h_j(x)) = \frac{1}{w}$,

$$\mathbb{E}[X_j] = \sum_{y \neq x} f(y)\, \mathbb{P}\big(h_j(y) = h_j(x)\big) = \frac{1}{w} \sum_{y \neq x} f(y) = \frac{N - f(x)}{w} \;\leq\; \frac{N}{w}.$$

Only the pairwise collision probability is needed; independence within a row is not required.

Meaning

On average, a row adds at most a $\frac{1}{w}$-share of the whole stream to the count of x.

The Model Behind the Guarantee

1. **Fixed stream.** The frequencies $f(y) \geq 0$ and $N = \sum_y f(y)$ are fixed before the hash functions are drawn.
2. **Random hashes.** $h_1, \ldots, h_d$ are drawn independently, and each satisfies $\mathbb{P}(h_j(x) = h_j(y)) = \frac{1}{w}$ for $x \neq y$.
3. **Non-negative updates.** Every update adds $+1$.

All randomness lies in the hash functions; the stream is not random.

Theorem: Point-Query Guarantee

Theorem (Count-Min Sketch error bound)

Let C be a Count-Min Sketch with d independent hash functions $h_1, \ldots, h_d$, each mapping items to w columns. For any fixed item x with true count $f(x)$, the estimate $\hat{f}(x)$ satisfies

$$f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N \quad \text{with probability at least } 1 - \delta,$$

provided

$$w = \left\lceil \frac{e}{\varepsilon} \right\rceil \quad \text{and} \quad d = \left\lceil \ln \frac{1}{\delta} \right\rceil,$$

where $N = \sum_y f(y)$ is the total number of items in the stream, ε is the error tolerance, and δ is the failure probability.

Proof: Lower Bound ($f(x) \leq \hat{f}(x)$)

Each counter $C[j, h_j(x)]$, $j = 1, \ldots, d$, is incremented every time x appears. Other items can only add to it, so

$$C[j, h_j(x)] = f(x) + X_j \;\geq\; f(x) \qquad \text{for every row } j.$$

Therefore the minimum across the d counters is at least $f(x)$:

$$\hat{f}(x) = \min_{1 \leq j \leq d} C[j, h_j(x)] \;\geq\; f(x) \quad \text{with probability 1.}$$

Proof: Upper Bound ($\hat{f}(x) \le f(x) + \varepsilon N$), One Row

The overestimate in row j is the collision error $X_j \ge 0$, with $\mathbb{E}[X_j] \le N/w$.

Markov's inequality

If $X \ge 0$ and $t > 0$, then $\mathbb{P}(X \ge t) \le \mathbb{E}[X]/t$.

How it is used: apply it to $X = X_j$ (non-negative) with threshold $t = \varepsilon N$:

$$\mathbb{P}(X_j > \varepsilon N) \le \mathbb{P}(X_j \ge \varepsilon N) \le \frac{\mathbb{E}[X_j]}{\varepsilon N} \le \frac{N/w}{\varepsilon N} = \frac{1}{\varepsilon w} \le \frac{1}{e},$$

because $w = \lceil e/\varepsilon \rceil \ge e/\varepsilon$. So each row is "bad" with probability at most $1/e$.

Proof: All Rows Together and Conclusion

The minimum exceeds $f(x) + \varepsilon N$ only if *every* row does:

$$\{\hat{f}(x) > f(x) + \varepsilon N\} = \bigcap_{j=1}^{d} \{X_j > \varepsilon N\}.$$

X_j depends only on h_j, and $h_1, \ldots, h_d$ are independent, so the probabilities multiply:

$$\mathbb{P}\big(\hat{f}(x) > f(x) + \varepsilon N\big) = \prod_{j=1}^{d} \mathbb{P}(X_j > \varepsilon N) \le \left(\frac{1}{e}\right)^{d} = e^{-d} \le \delta,$$

since $d = \lceil \ln \frac{1}{\delta} \rceil \ge \ln \frac{1}{\delta}$. Combining with the lower bound,

$$\mathbb{P}\big(f(x) \le \hat{f}(x) \le f(x) + \varepsilon N\big) \ge 1 - \delta. \qquad \blacksquare$$

Choosing the Width and the Depth

Width controls accuracy

One row fails with probability $\leq \frac{1}{\varepsilon w}$; we need this $\leq \frac{1}{e}$, i.e. $w \geq \frac{e}{\varepsilon}$.

$\varepsilon \downarrow \Rightarrow \;\; w \uparrow$

Depth controls confidence

d independent rows fail together with probability $\leq e^{-d}$; we need $e^{-d} \leq \delta$, i.e. $d \geq \ln \frac{1}{\delta}$.

$\delta \downarrow \Rightarrow \;\; d \uparrow$

Why e? With per-row failure bound $q = \frac{1}{\varepsilon w}$: $w = \frac{1}{\varepsilon q}$, $d = \frac{\ln(1/\delta)}{\ln(1/q)}$ (ignoring ceilings), so

$$dw = \frac{\ln(1/\delta)}{\varepsilon} \cdot \frac{1}{q \ln(1/q)},$$

and $q \ln(1/q)$ is largest at $q = 1/e$: this uses the fewest counters. At $q = 1/e$: $w = \frac{e}{\varepsilon}$, $d = \ln \frac{1}{\delta}$ and $dw = \frac{e \ln(1/\delta)}{\varepsilon}$, which is exactly our choice.

Memory Bound and Remarks

Using $w = O(\frac{1}{\varepsilon})$ and $d = O(\ln \frac{1}{\delta})$,

$$dw = O\Big(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\Big).$$

The number of counters depends on the desired accuracy and confidence, *not* on the number of distinct items.

Remarks on the guarantee

- **One item at a time.** δ is the failure probability of one fixed query. For all m items at once use a union bound: $d = \lceil \ln(m/\delta) \rceil$.
- **Oblivious stream.** The stream must not depend on the hash functions.
- **Markov is loose.** Real failure probabilities are usually far below e^{-d}.

Effect 1: Stream Length N

The guarantee is $f(x) \le \hat{f}(x) \le f(x) + \varepsilon N$. For a fixed sketch, a longer stream means a larger error bound.
Fixed sketch ($\varepsilon = 0.001,\ \delta = 0.01$, so $w = 2719,\ d = 5$):

N	error bound εN	average bucket load N/w
10^6	10^3	≈ 368
10^9	10^6	$\approx 3.7 \times 10^5$

To keep a fixed error E

Need $\varepsilon N \le E$, so $w = \lceil eN/E \rceil$. Example: $E = 100,\ N = 10^6$ gives $w = 27{,}183$.

Effect 2: Near-Unique Streams

Suppose almost every item occurs once: $f(x) \approx 1$ and the number of distinct items $m \approx N$. Then the N items spread evenly, so every bucket holds about N/w counts:

$$\hat{f}(x) \approx \frac{N}{w} \quad \text{for every } x, \text{ whether } f(x) = 1 \text{ or } 0.$$

Example: $\varepsilon = 0.001,\ w = 2719,\ N = 10^9$ gives every estimate $\approx 3.7 \times 10^5$.

Lesson

The theorem still holds, but εN is useful only when $f(x) \gg \varepsilon N$. Here the sketch cannot tell an item seen once from an item never seen. For rare-item frequencies, an exact table or another frequency-counting method may be more appropriate. Bloom filters are useful for membership queries, while HyperLogLog is useful for estimating the number of distinct items.

Effect 3: Number of Distinct Items m

Sketch with $\varepsilon = 0.001$, $\delta = 0.01$: $dw = 13{,}595$ counters ≈ 54 KB (4-byte counters), **whatever m is**.

distinct items m	exact table (4-byte counters)	Count-Min
10^4	40 KB	54 KB
10^6	4 MB	54 KB
10^9	4 GB	54 KB

Counter storage only; key/dictionary overhead of the exact table is ignored.

Break-even

The sketch is smaller only when $m > dw = 13{,}595$. For few distinct items an exact table is smaller *and* exact. The saving is real only when $m \gg dw$.

Effect 4: Order and Merging

- **Order does not matter.** $C[j,k] = \sum_t \mathbf{1}\{h_j(x_t) = k\}$ is a sum of one-hot updates, so any reordering of the stream gives the same table.
- **Sketches merge.** With the same d, w and hash functions, $C^{(S_1 \circ S_2)} = C^{(S_1)} + C^{(S_2)}$ entry by entry, and the guarantee holds with $N = N_1 + N_2$. Sketches built on separate machines can be combined.
- **Old data stays.** Counters never forget, so $\hat{f}(x)$ never decreases. For drifting streams use windows or periodic resets.

When the Stream Breaks the Assumptions

- **Deletions.** Allowed only if the net count of every item stays ≥ 0 at all times (then each counter is still $f(x)$ plus non-negative collision mass). If some net count can go negative, the guarantee $\hat{f} \geq f$ breaks and the minimum can underestimate (use Count Sketch).
- **Adaptive adversary.** A stream chosen after seeing the hashes can force collisions; keep seeds secret and draw them before the stream.
- **Weak or repeated hashes.** The proof needs $\mathbb{P}(h_j(x) = h_j(y)) = \frac{1}{w}$ and independent rows. Using one hash in every row makes all rows identical.
- **Counter overflow.** A 32-bit counter wraps at $2^{32} \approx 4.3 \times 10^9$ and breaks $\hat{f} \geq f$; use wider counters for long streams.

Worked Example: Setup

Stream ($N = 20$ arrivals, 7 distinct items):

$$A, B, A, C, A, B, A, D, A, C, E, A, B, A, F, D, B, C, A, G.$$

Step 1: choose the accuracy. Take $\varepsilon = 0.25$ and $\delta = 0.05$:

$$w = \left\lceil \frac{e}{0.25} \right\rceil = \lceil 10.87 \rceil = 11, \qquad d = \lceil \ln 20 \rceil = \lceil 2.996 \rceil = 3.$$

The sketch has $3 \times 11 = 33$ counters; row j uses hash function h_j. **Goal:** estimate $f(A)$ (frequent) and $f(G)$ (rare).

Honest remark

Seven distinct items need only 7 exact counters (counter storage only; key overhead ignored), fewer than 33. This is a toy run to show the mechanics; CMS saves memory only when the number of distinct items is much larger than dw.

Step 2: True Frequencies and Hash Values

For verification, the true frequencies of the items are calculated separately: A, B, C, D, E, F, and G occur 8, 4, 3, 2, 1, 1, and 1 times, respectively, giving $N = 20 = 8 + 4 + 3 + 2 + 1 + 1 + 1$. Before processing the stream, the hash functions are randomly chosen. Once chosen, they remain fixed throughout the algorithm:

Item	A	B	C	D	E	F	G
Frequency	8	4	3	2	1	1	1
h_1	1	5	7	10	4	3	7
h_2	3	8	7	3	1	8	8
h_3	9	11	4	10	9	7	4

For example, $A \xrightarrow{(h_1,h_2,h_3)} (1,3,9)$: arriving A increments $C[1,1]$, $C[2,3]$, $C[3,9]$.

Step 3: Processing Item 1 of 20 (A)

Hashes: $h_1(A) = 1$, $h_2(A) = 3$, $h_3(A) = 9$
Updates: $C[1,1]$: $0 \to 1$, $C[2,3]$: $0 \to 1$, $C[3,9]$: $0 \to 1$
Processed prefix: A

Sketch after item 1

	1	2	3	4	5	6	7	8	9	10	11
h_1	**1**	0	0	0	0	0	0	0	0	0	0
h_2	0	0	**1**	0	0	0	0	0	0	0	0
h_3	0	0	0	0	0	0	0	0	**1**	0	0

Step 3: Processing Item 2 of 20 (B)

Hashes: $h_1(B) = 5$, $h_2(B) = 8$, $h_3(B) = 11$
Updates: $C[1, 5]$: $0 \to 1$, $C[2, 8]$: $0 \to 1$, $C[3, 11]$: $0 \to 1$
Processed prefix: A, B

Sketch after item 2

	1	2	3	4	5	6	7	8	9	10	11
h_1	1	0	0	0	**1**	0	0	0	0	0	0
h_2	0	0	1	0	0	0	0	**1**	0	0	0
h_3	0	0	0	0	0	0	0	0	1	0	**1**

Step 3: Processing Item 3 of 20 (A)

Hashes: $h_1(A) = 1,\ h_2(A) = 3,\ h_3(A) = 9$
Updates: $C[1,1]$: $1 \to 2,\ C[2,3]$: $1 \to 2,\ C[3,9]$: $1 \to 2$
Processed prefix: A, B, A

Sketch after item 3

	1	2	3	4	5	6	7	8	9	10	11
h_1	**2**	0	0	0	1	0	0	0	0	0	0
h_2	0	0	**2**	0	0	0	0	1	0	0	0
h_3	0	0	0	0	0	0	0	0	**2**	0	1

Step 3: Processing Item 4 of 20 (C)

Hashes: $h_1(C) = 7,\ h_2(C) = 7,\ h_3(C) = 4$
Updates: $C[1,7]$: $0 \to 1,\ C[2,7]$: $0 \to 1,\ C[3,4]$: $0 \to 1$
Processed prefix: A, B, A, C

Sketch after item 4

	1	2	3	4	5	6	7	8	9	10	11
h_1	2	0	0	0	1	0	**1**	0	0	0	0
h_2	0	0	2	0	0	0	**1**	1	0	0	0
h_3	0	0	0	**1**	0	0	0	0	2	0	1

Step 4: Process the Remaining 16 Items

Continue with the same rule: for each arriving item, add 1 to $C[j, h_j(\cdot)]$ in all three rows.

$$A, B, A, D, A, C, E, A, B, A, F, D, B, C, A, G.$$

Two facts to notice:

- Every arrival adds exactly 1 to each row, so every row sum equals the number of arrivals processed.
- Addition is commutative, so the final table is identical for any reordering of the 20 arrivals.

Instead of storing every item, we only keep track of how many times each counter is updated, thereby saving memory

Final Sketch After All 20 Items

	1	2	3	4	5	6	7	8	9	10	11	Row sum
h_1	**8**	0	1	1	4	0	**4**	0	0	2	0	20
h_2	1	0	**10**	0	0	0	3	**6**	0	0	0	20
h_3	0	0	0	**4**	0	0	1	0	**9**	2	4	20

Green: For item A, we look at the three counters selected by its hash values $(1, 3, 9)$. These counter values give the frequency estimates for A.

Red:For item G, we look at the counters selected by $(7, 8, 4)$. These values are used to estimate the frequency of G.

Problem 1: A Frequent Item $(A,\ f(A) = 8)$

Read $C[1,1] = 8,\ C[2,3] = 10,\ C[3,9] = 9$:

$$\hat{f}(A) = \min\{8, 10, 9\} = 8 = f(A).$$

Where do the extra counts come from?

- Row 1: bucket 1 is used only by A: $X_1 = 0$.
- Row 2: $h_2(D) = h_2(A) = 3$, so D adds 2: $X_2 = 2$.
- Row 3: $h_3(E) = h_3(A) = 9$, so E adds 1: $X_3 = 1$.

$X_1 = 0,\ X_2 = 2,\ X_3 = 1$, so

$$\hat{f}(A) = f(A) + \min_j X_j = 8 + 0 = 8.$$

Here, taking the minimum reduces the effect of collisions and therefore reduces the chances of overestimating the true frequency.

Problem 2: A Rare Item (G, $f(G) = 1$)

Read $C[1,7] = 4$, $C[2,8] = 6$, $C[3,4] = 4$:

$$\hat{f}(G) = \min\{4, 6, 4\} = 4, \qquad f(G) = 1.$$

- Row 1: C shares bucket 7: $X_1 = 3$.
- Row 2: B and F share bucket 8: $X_2 = 4 + 1 = 5$.
- Row 3: C shares bucket 4: $X_3 = 3$.

Every row was contaminated, so the estimate is 4 instead of 1: absolute error $4 - 1 = 3$, relative error $\frac{4-1}{1} \times 100 = 300\%$, i.e. a 300% overestimate.

Lesson

The error depends on what else lands in the bucket, not on $f(x)$. Items with small true frequencies can be greatly affected by collisions, while items with large frequencies are less affected.

Checking the Example Against the Guarantee

With $\varepsilon = 0.25$, $N = 20$: $\varepsilon N = 5$. For a fixed item, with probability at least $1 - \delta = 0.95$ (over the random hashes) we guarantee $f(x) \le \hat{f}(x) \le f(x) + 5$; the lower bound always holds.

Item	$f(x)$	$\hat{f}(x)$	error	within $\varepsilon N = 5$?
A	8	8	0	yes
G	1	4	3	yes

Both estimates are inside the bound, but the guarantee is based on a fraction of N, so an error of 5 can be large compared to a rare item's true frequency.

Numerical Check 1: Expected Collision Error

Plug the example into $\mathbb{E}[X_j] = \frac{N - f(x)}{w}$ with $N = 20$, $w = 11$:

$$f(A) = 8: \quad \mathbb{E}[X_j] = \frac{20 - 8}{11} \approx 1.09 \qquad\qquad f(G) = 1: \quad \mathbb{E}[X_j] = \frac{19}{11} \approx 1.73.$$

The expectation is an average over random hash draws; one draw can land above or below it.

In our single draw

Realised errors: $0, 2, 1$ for A (average 1) and $3, 5, 3$ for G (average ≈ 3.67).

Numerical Check 2: Markov Bound and Multiple Rows

Threshold $t = \varepsilon N = 0.25 \cdot 20 = 5$. For A, $\mathbb{E}[X_j] \approx 1.09$, so

$$\mathbb{P}(X_j \geq 5) \leq \frac{1.09}{5} \approx 0.22.$$

Without knowing $f(A)$, use $\mathbb{E}[X_j] \leq N/w$:

$$\mathbb{P}(X_j > \varepsilon N) \leq \frac{1}{\varepsilon w} = \frac{1}{0.25 \cdot 11} \approx 0.364 \leq \frac{1}{e} \approx 0.368.$$

With $d = 3$ independent rows:

$$\text{General theorem bound:} \quad \mathbb{P}(\text{all three rows bad}) \leq \left(\frac{1}{e}\right)^3 \approx 0.0498 \leq \delta = 0.05,$$

$$\text{For this particular item } A: \quad 0.218^3 \approx 0.0104.$$

What the probability means

It is over the random choice of hash functions, made before the stream. The table above is one draw; the bound says nothing about one fixed table. Markov is loose: the realised errors for A are at most 2.

Application: Network Traffic Monitoring

Networks produce a never-ending stream of packets.

- Item x = an IP address.
- $f(x)$ = number of packets from that address.
- $\hat{f}(x)$ = estimate read from the sketch.

Why Count-Min Sketch?

We can estimate the number of packets associated with an IP address without keeping a counter for every address on the internet.

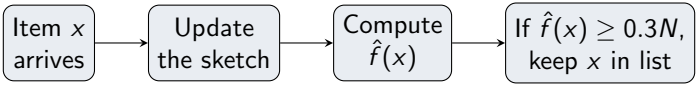
CMS estimates the frequency of tracked items; a separate candidate-generation method is needed to discover which items are heavy.

Application: Heavy Hitters

What is a heavy hitter?

An item that appears very often. Example: $f(x) \geq 0.3N$, i.e. at least 30% of the whole stream.

Problem: the sketch only answers "how many times is x?" It cannot tell us *which* items are heavy. So we add a small list of candidates.



At the end, the list holds our heavy hitters.

The candidates come from the arriving items themselves (each is checked on arrival) or from another discovery method; the sketch itself stores no item names, only the short list does.

Heavy Hitters: A Tiny Example

Stream: $A, A, A, B, C, A, B, D, A, A$ so $N = 10$, and the cut-off is $0.3N = 3$.

Item	A	B	C	D
$f(x)$	6	2	1	1

Only A has $f(A) \geq 3$, so A is the heavy hitter.

Why we never miss one

Since $\hat{f}(x) \geq f(x)$, a truly heavy item always passes the test $\hat{f}(x) \geq 0.3N$.

A light item can only sneak in if its overcount is large, and that overcount is at most εN (with probability at least $1 - \delta$). A small ε keeps wrong items out.

Application: Web and Search Analysis

Search engines receive a huge stream of queries.

- Item x = a search query, e.g. "weather".
- $f(x)$ = how many times it was searched.
- $\hat{f}(x)$ estimates how often a given query was searched over the monitored stream/window.

Why Count-Min Sketch?

Frequent queries are exactly where the sketch is accurate.

CMS estimates the frequency of tracked items; a separate candidate-generation method is needed to discover which queries are popular.

Application: Social Media

Platforms receive millions of posts every minute.

- Item x = a hashtag, e.g. `#AI`.
- $f(x)$ = how many times it was used.
- $\hat{f}(x)$ estimates how often a given hashtag occurred; using a time window or periodic reset, CMS can help track frequently occurring hashtags.

Why Count-Min Sketch?

The sketch stays the same small size however many posts arrive.

CMS estimates the frequency of tracked items; a separate candidate-generation method is needed to discover which hashtags are popular.

Application: Logs and Databases

Servers record every request they handle.

- Item x = an API endpoint.
- $f(x)$ = how many times it was called.
- $\hat{f}(x)$ estimates how often a given endpoint is used.

Why Count-Min Sketch?

We get useful statistics without storing the whole log history.

CMS estimates the frequency of tracked items; a separate candidate-generation method is needed to discover which endpoints are used most.

Advantages and Limitations

Advantages

- Memory efficient
- Fast updates and queries
- Single-pass processing
- Suitable for continuous streams
- Scalable
- Controlled probabilistic error

Limitations

- Approximate; collisions only overestimate
- Error εN is absolute: rare items are not reliable
- Near-uniform streams: useless
- Saves memory only if distinct items $\gg dw$
- Cannot list items without a candidate set
- Needs good hashes, an oblivious stream, and non-negative updates

Conclusion: From Exact Counting to a Compact Sketch

1. The Problem

Massive data streams make exact frequency counting expensive because the number of distinct items can be extremely large.

2. The Count-Min Sketch Solution

$$\text{Stream} \longrightarrow \text{Hash} \longrightarrow \text{Counters} \longrightarrow \text{Minimum} \longrightarrow \text{Frequency Estimate}$$

3. The Core Mechanism

For an item x, one counter is updated in every row, $C[j, h_j(x)] \leftarrow C[j, h_j(x)] + 1$, and a query returns $\hat{f}(x) = \min_{1 \le j \le d} C[j, h_j(x)]$.

Conclusion: Why It Works

1. What Makes It Reliable?

Collisions can only add extra counts, so for non-negative updates $\hat{f}(x) \geq f(x)$. Multiple independent rows make it unlikely that every row has a large collision error.

2. The Fundamental Trade-off

More counters give better accuracy and higher confidence:

$$w = \left\lceil \frac{e}{\varepsilon} \right\rceil, \quad d = \left\lceil \ln \frac{1}{\delta} \right\rceil \quad \text{and therefore} \quad \text{Memory} = O\left(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\right).$$

Conclusion: Final Takeaway

Final Takeaway

Count-Min Sketch does not try to remember the entire stream. It remembers just enough information to answer frequency queries quickly, using very little memory, with a mathematically controlled error.

The Catch

The error is measured against N: estimates are reliable for frequent items and uninformative for rare ones.

References

1. Video lectures (YouTube playlist):
   youtube.com/playlist?list=PL2mpR0RYFQsAR5RyB54FyEE9vUiGtCSZM
   youtube.com/playlist?list=PLM_KIlUOWoXmkV4QBa1Dg8PtJaHTdWHwRS
   youtube.com/watch?v=ibxXO-b14j4
2. Lecture notes (Stanford): theory.stanford.edu/~tim/s15/l/l2.pdf
3. Problem set 2 (MIT 6.854): people.csail.mit.edu/moitra/docs/6854hw2.pdf
4. Preprint (EasyChair): easychair.org/publications/preprint/gNlw/open
5. Lecture 7 notes (IIT Bombay, CS 602):
   cse.iitb.ac.in/~nutan/courses/cs602-13/notes/lec7.pdf
6. Scribe notes (Rice University, COMP 480/580, Fall 2024):
   cs.rice.edu/~as143/COMP480_580_Fall24/scribe/scribe11.pdf

Use of AI

AI (Claude, by Anthropic) was used to make this presentation (slides and LaTeX source) and to run sanity checks on the worked example, calculations and formulas.

Thank You

Questions & Discussion

Daksh · Shaurya · Swapnil · Tridham · Akshay · Shinjan G
Indian Statistical Institute